SISTEMA BANCARIO POR INTERNET BP

Arquitectura Propuesta

Diagrama Contextual

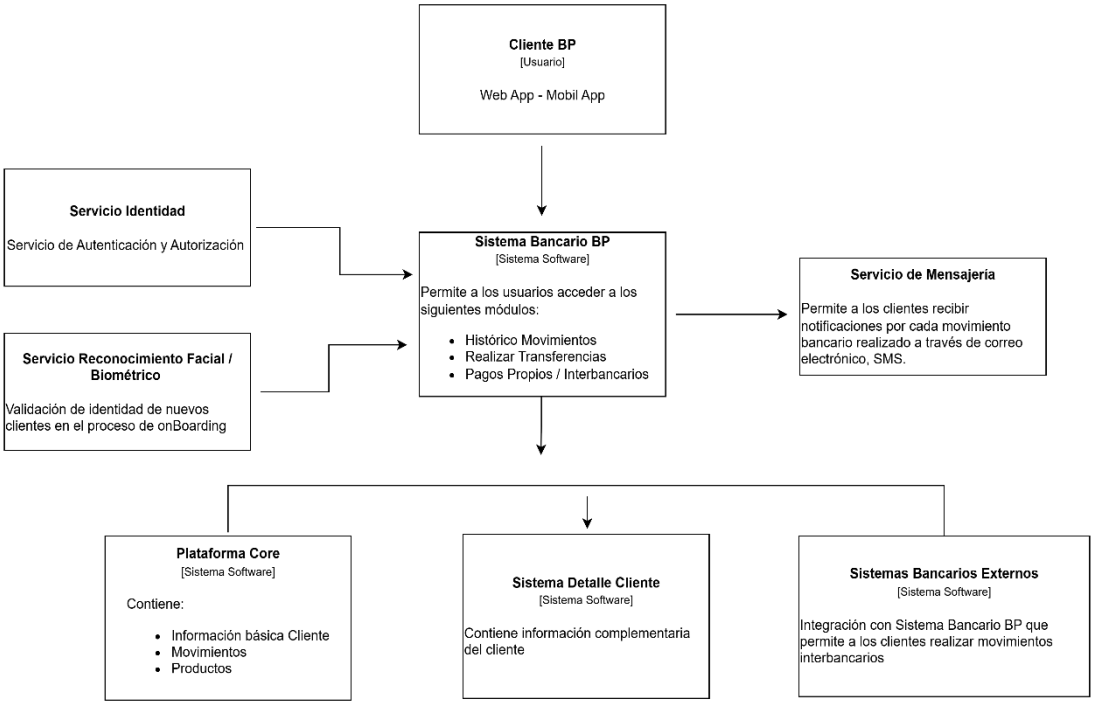


Diagrama de Contenedor

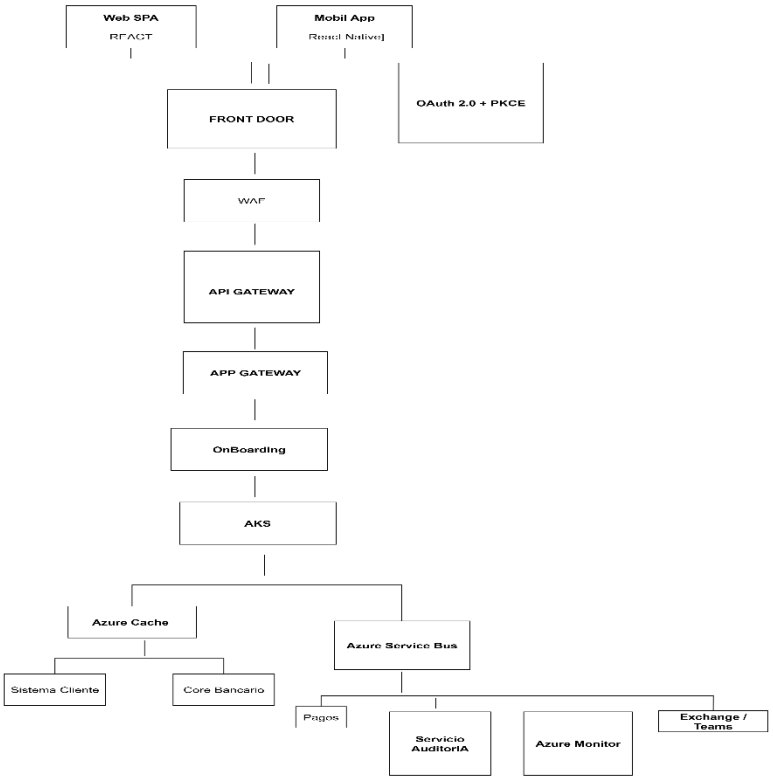
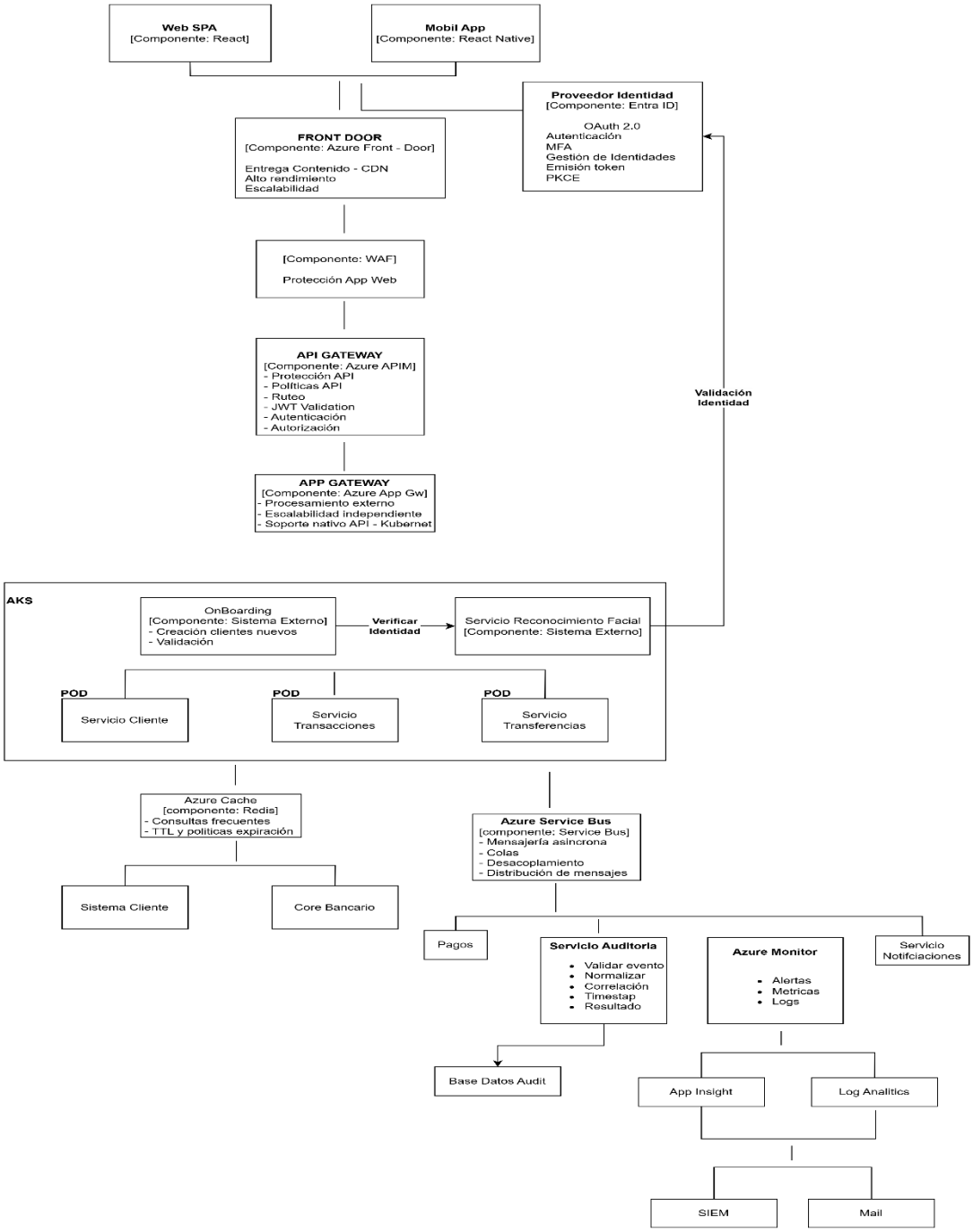


Diagrama de Componente



Decisión

La presente arquitectura se propone implementar en la nube de Azure, la misma que brinda todas las prestaciones para cumplir con una plataforma robusta a nivel de acceso, seguridad, alta disponibilidad, monitoreo, observabilidad y auto gestión de los componentes parte de esta solución, las mismas que se ajustan a tecnologías de última generación.

¿Por qué usar Azure?

La nube de Azure dispone de todos los componentes y servicios requeridos como parte de la solución propuesta, los mismos que son auto gestionados, es decir se requiere un mínimo de intervención humana para la operación.

Los servicios y componentes a utilizar son los siguientes:

- Azure Front Door: Entrada de peticiones por parte del cliente
- Azure WAP: Seguridad en la capa de aplicación Web
- Azure API Management: Seguridad, políticas
- AKS: Microservicios
- Azure Service Bus: Mensajeria
- Azure Event Grid: Eventos
- Azure SQL: Base de datos relacional
- Azure Cache: Redis
- Azure Key Vault: Credenciales
- Entra ID: Identidades
- Azure Monitor y App Insights: Monitoreo
- Log Analytics: Logs
- Front Door: CDN
- DR: Azure Region
- Azure Storage: Storage

Análisis Servicios

- **Front-End**

  El requerimiento indica disponer de un SPA y una aplicación móvil que sea multiplataforma.

  Alternativas Solución:
  - React
  - Angular

  Solución Escogida: Framework React
  - Es una solución multiplataforma
  - Permite construir componentes reutilizables
  - Comparte conceptos de IU entre diferentes módulos.
- **Aplicación Móvil**

  Alternativas Solución:
  - React Native
  - Flutter

Solución Escogida: Framework React

- Reutilización de componentes con el Front-End.

- **Autenticación OAuth 2.0**

  Alternativas Solución:

  - Authorization Code + PKCE
  - Implicit flow

  Solución Escogida: Authorization Code + PKCE

  - Se ajusta para la aplicación Web y Movil
  - El estándar actual de OAuth recomienda usar PKCE.

- **Microservicios Principales**

  Lo dividimos en los siguientes de acuerdo al requerimiento:

  - Servicio al Cliente (Información básica, productos, cuentas, información complementaria)
  - Servicio Transacción (Movimientos, historicos)
  - Servicio Transferencia (Transferencias, Pagos)

- **Service Bus**

  Para el servicio de transferencias no se debe considerar peticiones HTTP síncronas, puesto que afectaría el desempeño del procesamiento de la información

  Peticiones Síncronas las utilizaremos para autenticación, autorización, consulta de saldos.

  Peticiones Asincronas las utilizaremos para procesamiento de transferencias, notificaciones, auditoria, eventos.

- **Auditoría**

  Como parte del requerimiento es necesario contemplar una base de datos para el servicio de auditoria.

  Se recomienda separar las bases de datos transaccionales de la requerida para auditoría.

- **Persistencia del cliente**

  El requerimiento indica mecanismo de persistencia de información para clientes frecuentes.

Solución: Para cumplir con este requerimiento se propone implementar Redis, el mismo que evita latencias, sobrecarga sobre el core bancario, cantidad de consultas recursivas y costos operativos.

- **Notificaciones**

  El requerimiento indica mínimo 2 soluciones para el servicio de notificaciones.

  Solución: Para cumplir con este requerimiento se propone notificaciones por "**Mail"** y "**SMS".**

- **API Gateway**

  A través del API Gateway se permite el acceso seguro a los servicios mediante un API Rest.

  Para esta propuesta consideramos la utilización de Azure API Management, cuyas prestaciones son las siguientes:

  - Autenticación
  - Autorización
  - Validación JWT (JBOSS Web Token)
  - Ruteo
  - Versionamiento
  - Politicas API
  - Protección API

- **Patrones Arquitectónicos**

  Como patrones arquitectónicos se consideran los siguientes:

  - API Gateway
  - Microservicios
  - Event – Driven Service
  - CQRS
  - Cache – Redis

- **Alta Disponibilidad**

  Para garantizar la operación y acceso al sistema bancario se propone brindar el servicio de alta disponibilidad considerando los siguientes puntos:

  - Zonas de Alta Disponibilidad
  - Instancias multiples
  - Auto escalamiento
  - Balanceadores de carga
  - Geo replicación
  - Respaldos
  - Failover

- **Disaster Recover - DR**

  Se propone medir:

  - RTO

    - RPO
- Estas variables se deben ajustar de acuerdo a los SLA definidos con el negocio.

- **Observabilidad**

  Se consideran los siguientes puntos importantes:

    - Logs (Azure Monitor – Log Analytics)
    - Métricas (Mem, CPU, errores, latencia, throughput, conexión base datos)
    - Traces (App Insight)

- **Monitoreo**
  Desarrollo de dashboard correspondientes a los siguientes dominios:

  Negocios

    - Transferencias por min
    - Pagos
    - Transferencias exitosas
    - Transferencias erróneas

  Técnica

    - Mem
    - Proc
    - Latencia

- **Normativa**

  En Ecuador, los elementos normativos y marco regulatorio para las entidades financieras tanto privadas como públicas están estructuradas a través de un conjunto jerárquico de leyes, juntas reguladoras y organismos de control, entre las cuales se consideran las de mayor relevancia:

    - **Código Orgánico Monetario y Financiero (COMF):** Es la ley máxima que rige el sistema monetario, financiero, de valores y seguros en el país. Regula la constitución, funcionamiento, actividades y liquidación de las entidades. [1, 2]
    - **Ley Orgánica de Prevención, Detección y Combate del Delito de Lavado de Activos y de la Financiación de otros Delitos:** Regula las obligaciones de cumplimiento y reporte de las instituciones financieras.
    - **Codificación de las normas de la Superintendencia de Bancos.**

- **Costos**

  Para optimización de costos se propone la implementación de serverless y servicios administrados a nivel de los siguientes servicios:

    - API Management
    - Service Bus
    - Azure SQL

- Redis
- Container Apps

Adicionalmente, como parte de la optimización de costos se recomienda incluir el servicio de Autoscaling, el cual permite ajustar los recursos de acuerdo a la demanda de uso de los mismos.

Se recomienda NO considerar máquinas virtuales.

- **Decisiones Arquitectónicas y Justificación**
  Las decisiones tomadas para la implementación de esta arquitectura está basado en las siguientes justificaciones por cada servicio contemplado

  - Microservicios: Separación de responsabilidades y despliegue independiente
  - API Gateway: Centralización de la seguridad, ruteo y versionamiento de API
  - Service Bus: Desacoplamiento y resilencia
  - Redis: Menos latencia y sobrecarga al core
  - OAuth + PKCE: Protección contra robo
  - React: Reutilización de componentes
  - React Native: Multiplataforma y reutilización de conocimientos.
  - Azure: Servicios administrados
  - CQRS: Separación de procesos de lectura y ecritura
  - WAF: Protección de aplicación Web
  - Multiregión: Continuidad de servicios y DR.

COMENTARIOS

- Se plantea una arquitectura robusta que cumple con todos los componentes tecnológicos, la misma que permita que los servicios implementados se ejecuten de manera eficiente, garantizando su seguridad, estabilidad, redundancia.
- Las decisiones adoptadas se apalancan en mejores prácticas de fabricante y la experiencia en mi trayectoria profesional, tanto en ambientes On-Premise y Cloud.
- Se adjunta a la presente el repositorio donde se encuentra el desarrollo del presente proyecto

https://github.com/hesalazar/Arquitecto_Soluciones.git